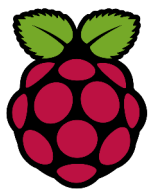


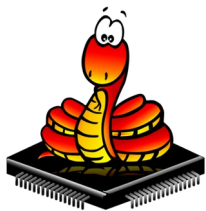


Atelier Raspi

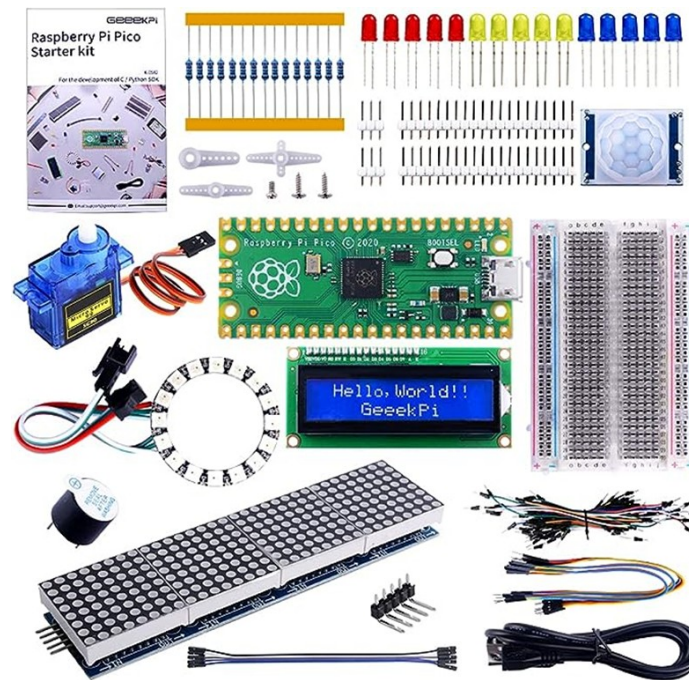
Atelier N°17 La mini serre instrumentée Module 4 relais



Logo du Raspberry Pico



Logo du MicroPython

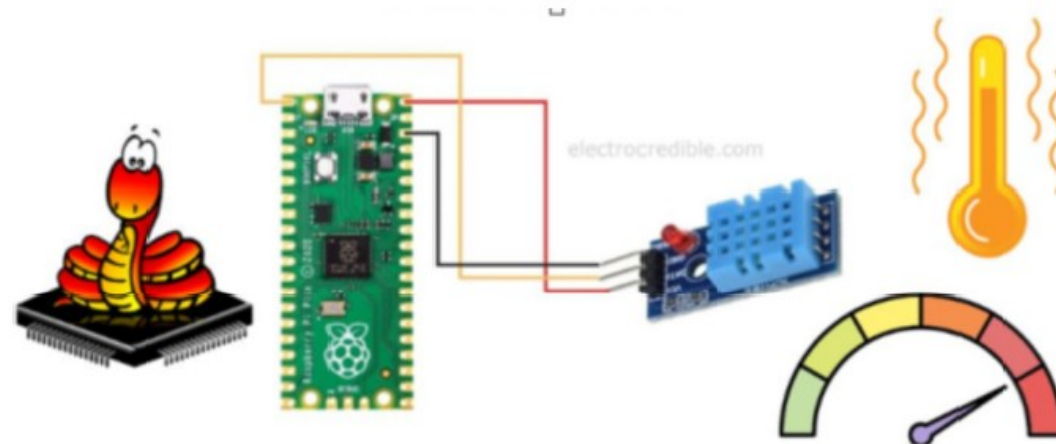


L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

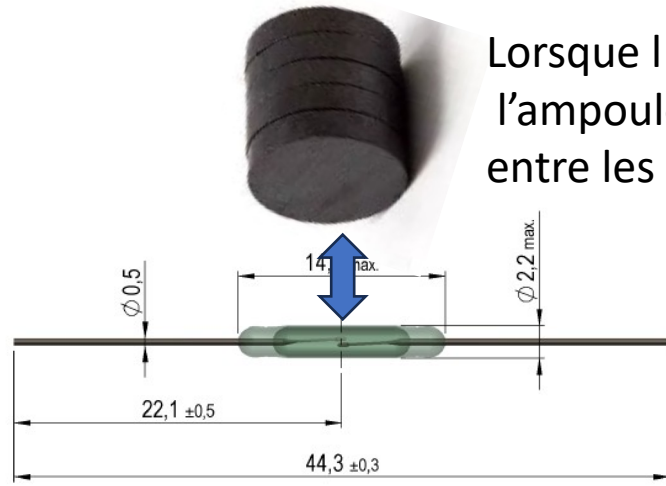
Atelier N°16 La mini serre instrumentée

Module 4 relais

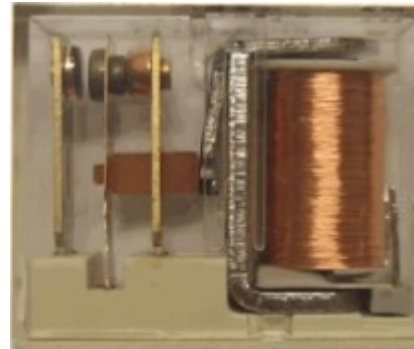
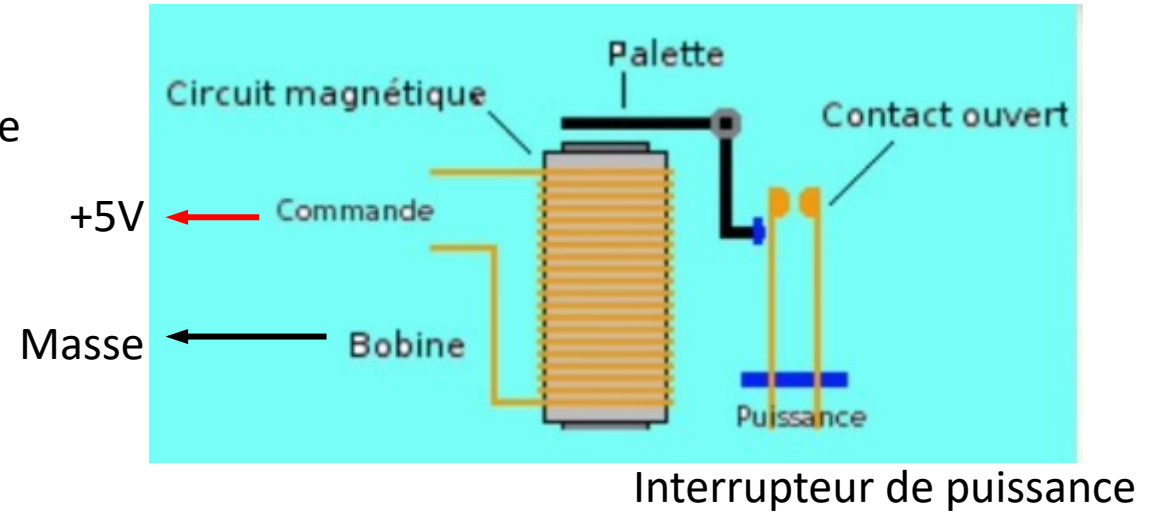
- Etape 1 Les relais
- Etape 2 Le câblage de la carte 4 relais
- Etape 3 les composants du bocal
- Etape 4 le brumisateur1/2
- Etape 5 le brumisateur2/2
- Etape 6 le logiciel
- GI1 Glossaire informatique1
- GI2 Glossaire informatique2
- GI3 Glossaire informatique3
- GI4 Glossaire informatique4
- GI5 Glossaire informatique5
- GI6 Glossaire informatique6
- GE1 Glossaire électronique



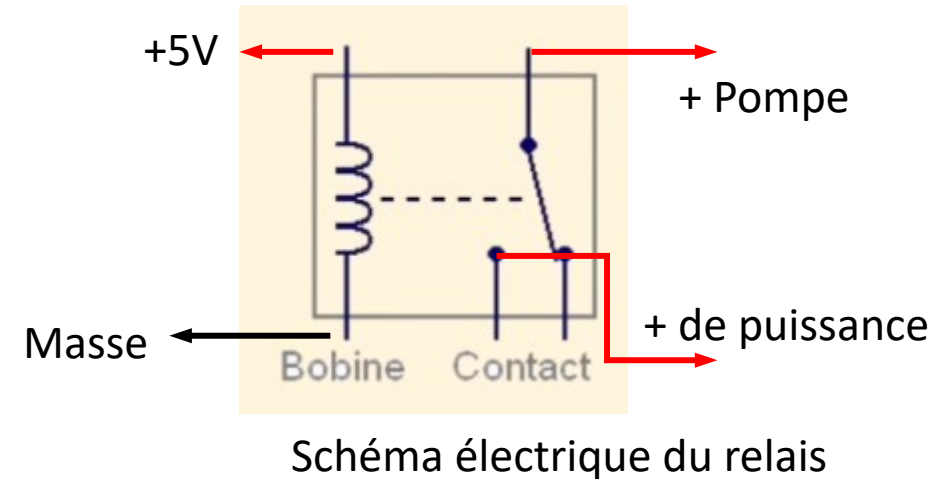
A17E1 Les relais

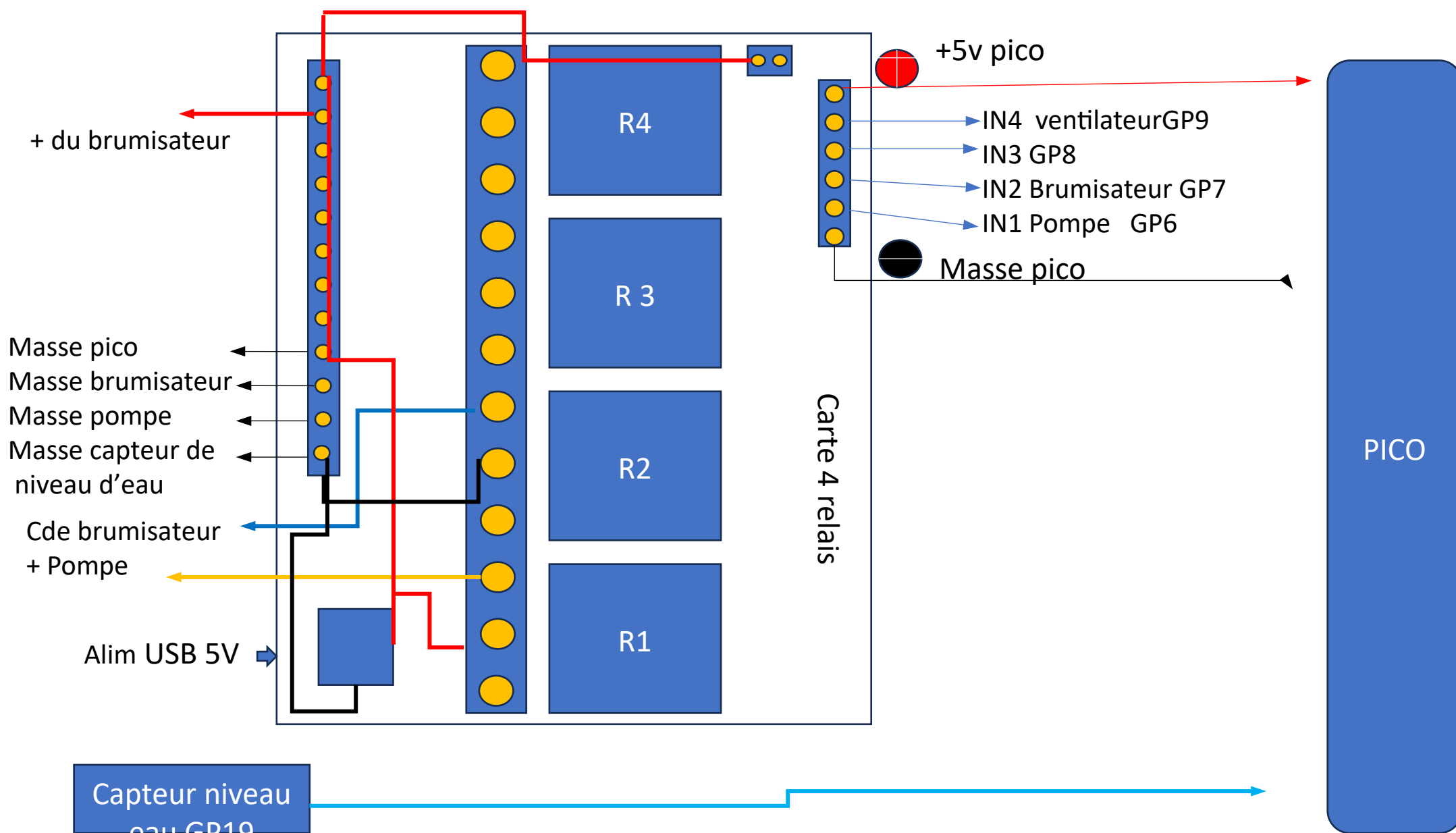


Lorsque l'aimant est proche de l'ampoule Reed, le contact se ferme entre les deux lamelles.

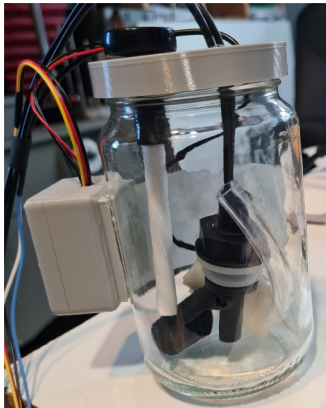


Vue technologique





A17E2 La carte 4 relais et câblage



Bocal avec le capteur de niveau d'eau, le brumisateur, la pompe à eau pour arrosage et l'électronique de commande du brumisateur dans la boîte grise

Carte pico avec IHM,
carte 4 relais et le bocal



Brumisateur piezzo



Pompe pour arrosage

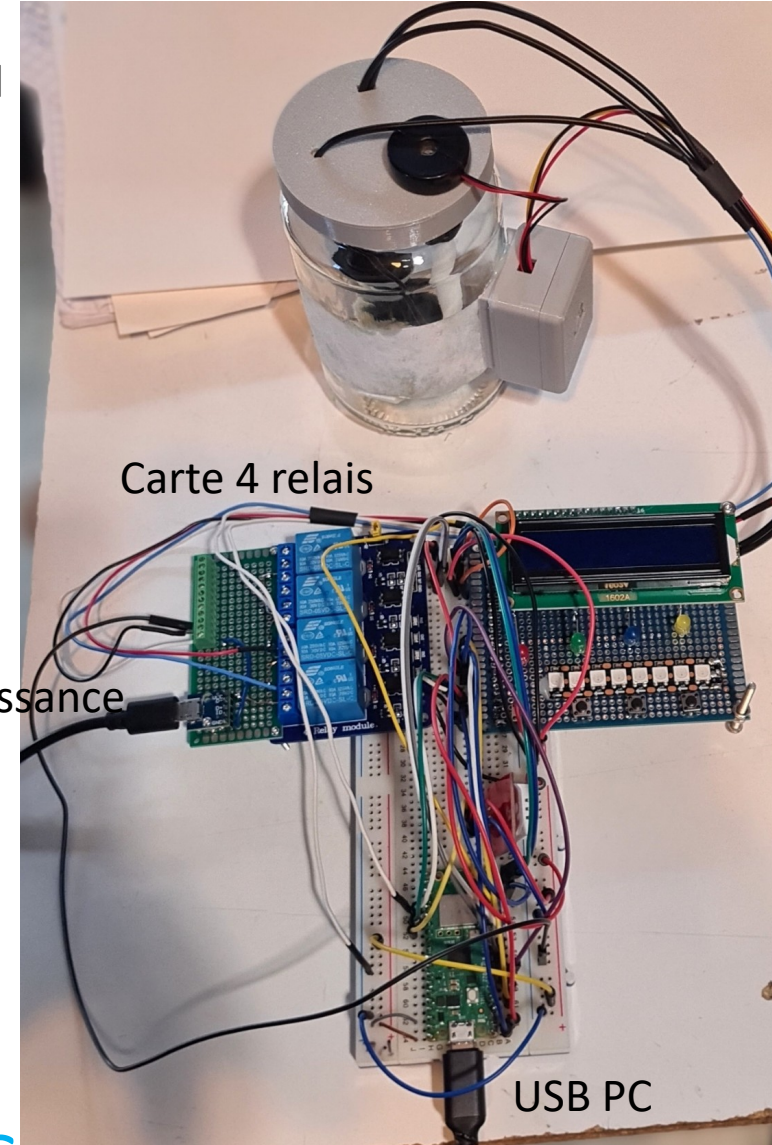
Capteur de niveau



Avec eau le flotteur remonte



Sans l'eau le flotteur descend



Carte 4 relais

Carte IHM

USB alim puissance

USB PC

A17E3 Les composants

● A17E4 Le brumisateur 1/2

Un **brumisateur piézoélectrique** transforme l'eau en une brume fine grâce à un **cristal piézoélectrique** à une fréquence ultrasonique. Les principales étapes du processus sont les suivantes :

1. Excitation électrique

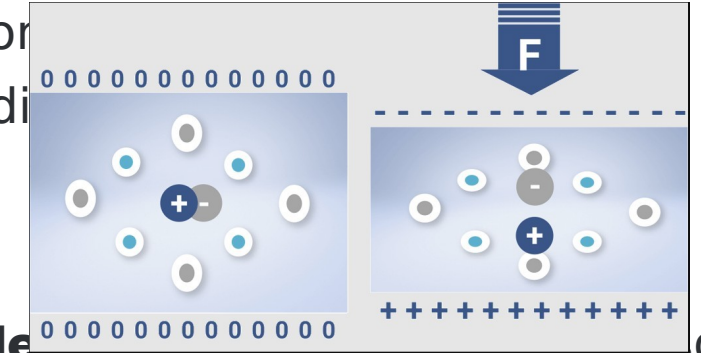
Un signal sinusoïdal est appliqué au cristal *PZT* (Titano-Zirconate de Plomb). Sous cette excitation, le cristal se déforme en se contractant et en se dilatant à la fréquence du signal, créant ainsi une vibration mécanique.

2. Génération d'ondes de Faraday

La vibration du cristal, placée sous la surface de l'eau, génère des **ondes de Faraday**. Ces ondes sont périodiques qui se forment lorsque la fréquence dépasse un seuil critique, provoquant l'instabilité de la surface de l'eau.

3. Atomisation de l'eau

Ces ondes provoquent un phénomène appelé cavitation : de minuscules bulles de vapeur se forment et implosent rapidement. L'implosion de ces bulles projette de fines gouttelettes d'eau dans l'air, formant ainsi la brume caractéristique du brumisateur. Ces ondes de Faraday arrachent des molécules d'eau (et éventuellement de petites quantités d'huiles présentes dans l'eau) de la surface, les pulvérisant en **gouttelettes très fines** (souvent de l'ordre de quelques microns).



A17E5 Le brumisateur 2/2

4. Formation de la brume

Les gouttelettes créées restent en suspension dans l'air, formant une **brume sèche** qui recouvre
La brume se disperse doucement, créant un effet visuel et humidifiant l'atmosphère

5. Caractéristiques supplémentaires

- **Fréquence** : typiquement autour de 1,65 MHz, ce qui place le son dans la gamme des ultrasons (le rend inaudible pour l'humain).

En résumé, le brumisateur piézoélectrique fonctionne grâce à la **vibration ultrasonique** d'un cristal qui crée des ondes de Faraday et pulvérise l'eau en micro-gouttelettes, produisant ainsi une brume

```
1 # On va commander la pompe et la led verte avec BP1 puis
2 # éteindre la pompe et la led avec BP2
3 from machine import Pin
4 import time
5 from quant_debounce_bp import Quant_Debounce_Bp
6 time.sleep(0.1) # Wait for USB to become ready
7 print("Test Pompe !")
8 # INIT
9 bp1_allumer = Quant_Debounce_Bp(0, "bp1")
10 bp2_eteindre = Quant_Debounce_Bp(1, "bp2")
11 led_etat = Pin(2, Pin.OUT) #Led verte
12 relais1_pompe = Pin(6, Pin.OUT)
13 # place les actionneur dans leur etat par default
14 relais1_pompe.value(1) # eteint la pompe
15 led_etat.value(1) # eteint la led
16 mon_etat = "eteint" # variable d'etat du programme
17 # boucle principale
18 while True:
19     if mon_etat == "eteint":
20         if bp1_allumer.get_state() == "appuyer":
21             # on allume la pompe
22             relais1_pompe.value(0)
23             led_etat.value(0)
24             mon_etat = "allumer"
25     else:
26         if bp2_eteindre.get_state() == "appuyer":
27             # on eteint la pompe
28             relais1_pompe.value(1)
29             led_etat.value(1)
30             mon_etat = "eteint"
```

A17E5 Le logiciel de commande de la pompe

La détection des BP se fait par l'import de:

Quant_Debounce_Bp.py qui est déjà sur le Pico


```

1 # On va commander le brumisateur et led rouge avec BP3
2 #et les éteindre avec BP4
3 from machine import Pin
4 import time
5 from quant_debounce_bp import Quant_Debounce_Bp
6
7 time.sleep(0.1) # Wait for USB to become ready
8
9 print("Test Brumisateur !")
10
11 # INIT
12 bp3_allumer = Quant_Debounce_Bp(11, "bp3")
13 bp4_eteindre = Quant_Debounce_Bp(10, "bp4")
14 led_etat = Pin(4, Pin.OUT) #LED rouge
15 relais2_brumisateur = Pin(7, Pin.OUT)
16
17
18 # place les actionneur dans leur etat par default
19 relais2_brumisateur.value(1) # eteint le brumisateur
20 led_etat.value(1)           # eteint la led
21 mon_etat = "eteint"         # variable d'etat du programme
22
23 # boucle principale
24 while True:
25     if mon_etat == "eteint":
26         if bp3_allumer.get_state() == "appuyer":
27             # on allume le brumisateur
28             relais2_brumisateur.value(0)
29             time.sleep(0.05)
30             relais2_brumisateur.value(1)
31             led_etat.value(0)
32             mon_etat = "allumer"

```

```

33 else:
34     if bp4_eteindre.get_state() == "appuyer":
35         # on eteint le brumisateur
36         relais2_brumisateur.value(0)
37         time.sleep(0.1)
38         relais2_brumisateur.value(1)
39         time.sleep(0.1)
40         relais2_brumisateur.value(0)
41         time.sleep(0.1)
42         relais2_brumisateur.value(1)
43
44         led_etat.value(1)
45         mon_etat = "eteint"

```

A17E5 Le logiciel
de commande du brumisateur

G|1 Glossaire informatique1

Instruction du langage : ne nécessite pas d'importer de module pour les utiliser

print('Bonjour') : affiche Bonjour sur la console du PC

While test : Boucle tant que test est VRAI

import : pour importer des modules contenant des fonctions

import time : permet d'appeler les fonctions relatives au temps

time.sleep(x) : arrête l'exécution du programme pendant x secondes

import machine : module contenant les fonctions permettant d'agir sur les périphériques du pi pico

from : pour simplifier l'écriture on importe certaines fonctions directement dans une variable à l'aide de « from »

from machine import Pin : permet d'accéder à la fonction Pin du module machine uniquement.

ma_led_verte = Pin(n, Pin.OUT) : permet d'indiquer comment on souhaite utiliser un GPIO,

ici on a configuré le p I/O N° n en sortie tout ou rien

ma_led_verte.Toggle() : inverse l'état de l'I/O associé à la variable.

Ici l'IO n passera de l'état haut (+3.3v) à l'état bas (0v) ou inversement

: En début de ligne permet d'écrire un commentaire, ce qui est écrit n'est pas une instruction et donc n'est pas pris en compte par le programme.

Variable **A=A+1** -> Nouvelle valeur = ancienne valeur+1 , peut s'écrire **A+=1**

print (" texte " , A), input ("texte" , B)

Opérateurs: +,-,*,/,%, **Données**: int(), float(), str(), bool()=True or False

G|2 Glossaire informatique2

Liste -> Création d'une liste, `liste_de_noms = ['Toto', 'Hubert', 'Jacques', 'Philippe']`

`liste_quelconque = ['Toto', 1, 'Philippe', 4,5,6]`

`print ('Nombre d'articles dans la liste : ', len(liste_quelconque))`

`for items in liste_quelconque:`

`print (items)` -> parcourt la liste et imprime chaque item de la liste

Pour récupérer un élément de la liste, **variable= liste[i]**, i étant l'indice d'ordre dans la liste

Dictionnaire-> liste d'éléments sous la forme d'une paire key:valeur.

`N = {'voiture':'4 roues', 'moto':'2 roues'}`

On accède à la valeur d'un key en l'utilisant comme index (clé de recherche):

`print(N['voiture'])` -> '4 roues'

Fonction-> `def nom_de_ma_fonction(args) :`

on peut ensuite l'appeler dans un programme avec: `nom_de_ma_fonction()`

Module-> `from machine import Pin` -> importe la fonction Pin du module machine

`P0 = Pin(0, Pin.OUT)` -> on paramètre le GPIO 0 en sortie

`P0.value(1)` -> on lui donne la valeur 1 ou état « haut »

G|3 Glossaire informatique 3

Boucles

for degree in range(0,180,1):

XXXXX -> la variable degree va varier de 0 à 180 avec un pas de 1, pour chaque valeur de cette variable, XXXX sera exécuté

global ma_variable -> permet d'avoir accès à ma_variable du programme principal dans la def d'une fonction

def ma_fonction():

global ma_variable

print(ma_variable)

ma_variable += 2

return ma_variable

Return ma_variable -> permet de donner au reste du programme la nouvelle valeur de ma_variable ou d'une nouvelle variable si créée dans la fonction

G|4 Les fonctions du module time

time.time() -> donne le temps présent en ms

time.time_diff(ticks1, ticks2) -> donne la différence entre ticks1 et 2

time.sleep (2) -> met un délai de 2 secondes

utime travaille comme time mais en ms

utime.sleep (100) -> met un délai de 100ms

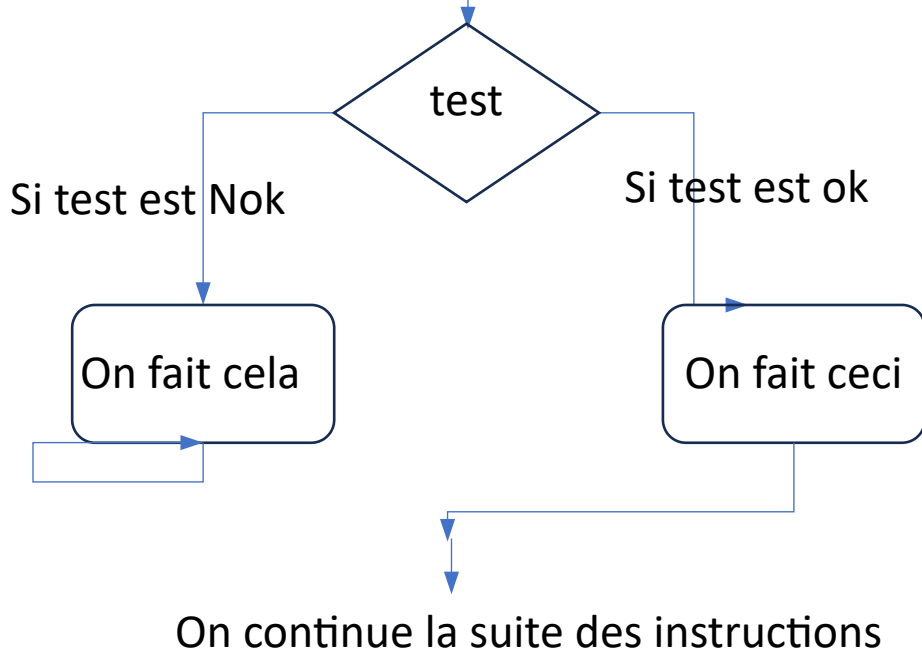
G|5 Glossaire informatique 3

Les tests:

l'instruction **if...else** permet d'exécuter un bloc de code en fonction de la véracité d'une condition, où le corps de l'instruction **if** s'exécute si la condition est **vraie**, et le corps de l'instruction **else** s'exécute si elle est **fausse**, avec une indentation pour définir les blocs de code.

Dans la suite des instructions du programme,

On arrive à un test



Exemple:

Dans le programme A=2

If A > 10:

 print('A supérieur à 10')

else:

 print ('A inférieur à 10')

On continue d'exécuter les instructions qui suivent

Gl6 Les fonctions du module PWM

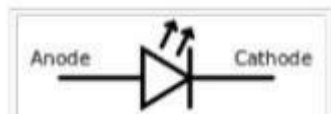
Fonctions utilisées:

```
servoPinPan = PWM(Pin(16))          # servoPinPan est un objet, initialise le Pin GPIO 16 en sortie PWM  
servoPinPan.freq(50)                # Initialise la fréquence des signaux carrés à 50Hz soit 20ms  
servoPinPan.duty_u16(int(newDuty))  # duty_u16 est le rapport cyclique entre le temps du signal carré haut sur  
sa période, new duty est une variable, le max est à 65535.
```

```
servoPinPan.deinit()                # réinitialise le PWM
```

```
servoX.duty_ns(pulseDX) #commande le rapport cyclique du PWM en nanosecondes suivant la variable  
pulseDX de l'objet servoX
```

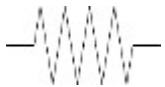
GE1 Glossaire électronique



Symbole de la diode LED



Pile



Résistance



BP

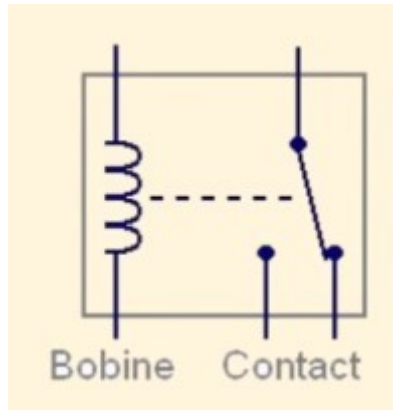
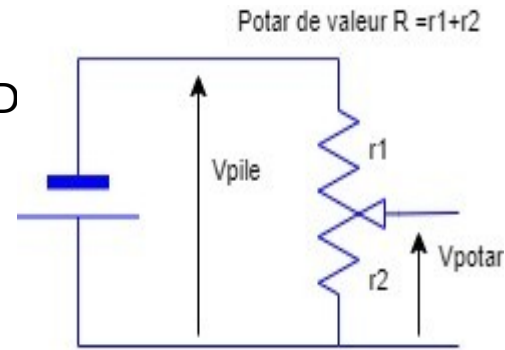


Schéma électrique du relais



$$V_{potar} = V_{pile} * r2 / (r1 + r2)$$



Condensateur
de filtrage signal



Condensateur
Chimique, réserve d'énergie

Couleurs	Tension de seuil ou Vf	If (mA)	Longueur d' onde
Rouge	1,6 V à 2 V	6à20	650 à 660 nm
Jaune	1,8 V à 2 V	6à20	565 à 570 nm
Vert	1,8 V à 2 V	6à20	585 à 590 nm
Bleu	2,7 V à 3,2 V	6à20	470 nm
blanc	3,5 v à 3,8 v	30	

